DA3408 Assignment 3

M Shivaarchitha Vudutha

October 5, 2026

Abstract

This report provides an empirical performance evaluation comparing Apache Spark (v3.5.1) and Ray Data (v2.59.0) on an identical distributed data preprocessing pipeline. The benchmark was conducted on the full ∼2.0 GB New York City Yellow Taxi Parquet dataset (36 monthly partitions spanning 2021–2023, comprising tens of millions of records) on an isolated 2-worker cluster topology.

---

1 Introduction and Theoretical Motivation

Traditionally, data pipelines rely on JVM-based Apache Spark for preprocessing, while modern AI models (PyTorch, JAX) run natively in Python. This split forces expensive cross-runtime data transfers and socket serialization (Py4J) whenever custom Python logic is applied. Ray Data resolves this through **Unified Compute** by executing natively in Python with zero-copy Apache Arrow streaming. Here, we benchmark both engines under identical 2-worker cluster conditions on ∼2.0 GB of uncleaned NYC Taxi trip data.

2 Cluster Orchestration (25 Points)

To ensure an objective comparison, both clusters were deployed on an isolated Linux environment (Ubuntu 24.04 LTS, Python 3.10.14 via pyenv, OpenJDK 17) with dedicated 2-worker compute slots.

2.1 Spark Standalone Cluster Topology

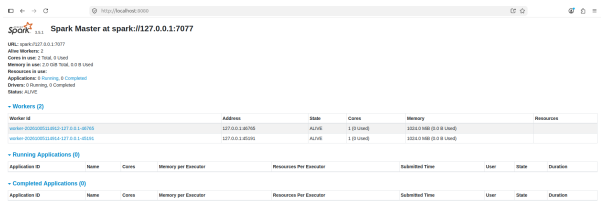
A Standalone Spark cluster was configured using `spark-3.5.1-bin-hadoop3`:

- **Master**: Bound to `spark://127.0.0.1:7077`, web UI accessible at port 8080.
- **Workers**: Two isolated worker daemon instances were launched using independent instance flags and UI ports:
  - Worker 1: 1 Core, 1 GB RAM (UI port 8081).
  - Worker 2: 1 Core, 1 GB RAM (UI port 8082).

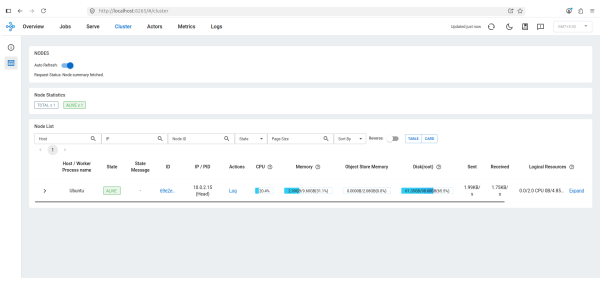
2.2 Ray Cluster Topology

A Ray cluster was instantiated using Ray Data 2.59.0:

- **Head Node**: Initialized with `ray start --head --num-cpus=2` to allocate exactly 2 CPU worker slots.
- **Object Store**: Configured with in-memory Plasma shared memory for zero-copy IPC.
- **Dashboard**: Monitored in real time on port 8265.

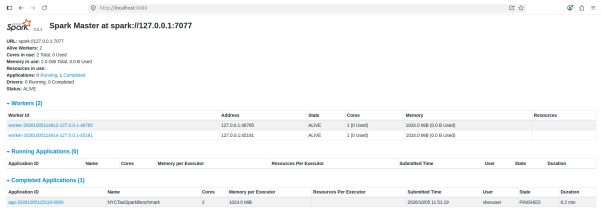


(a) Spark Master UI verifying 2 Alive Workers and 2.0 GiB memory.

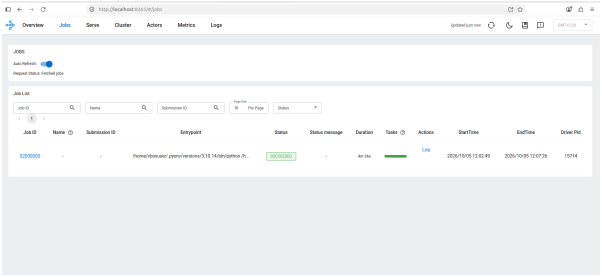


(b) Ray Dashboard confirming 2 active CPU workers on the head node.

Figure 1: Cluster orchestration and worker allocation verification across both engines.



(a) Spark UI showing completed application `NYCTaxiSparkBenchmark`.



(b) Ray Dashboard tracking live task pool operator execution.

Figure 2: Runtime execution confirmation on Spark (Port 4040/8080) and Ray (Port 8265).

3 Pipeline Parity and Transformations (25 Points)

Both pipelines execute identical business logic across the 36-month dataset:

1. **Ingestion**: Read 36 partitioned Parquet files (`yellow_tripdata_2021-*.parquet` through `2023-*.parquet`, ∼2.0 GB total) and auxiliary lookup table `taxi_zone_lookup.csv`.

2. **Cleansing**: Eliminate records with missing pickup/dropoff timestamps, drop rows with null `PULocationID`, and purge invalid non-positive distances ($trip_distance \leq 0$).

3. **Heavy Join**: Execute an inner join between taxi trip records and zone metadata on:

$$PULocationID = LocationID$$

   enriching records with Borough, Zone, and Service Zone identifiers.

4. **Feature Engineering (UDF)**: Calculate trip speed in miles per hour:

$$avg_speed_mph = \frac{trip_distance}{(tpep_dropoff_datetime - tpep_pickup_datetime)_{hours}}$$

   filtering anomalous trip durations (≤ 0 or > 24 hours).

5. **Export**: Write structured output to partitioned Parquet files ready for downstream training.

3.1 Performance Tuning Note (Attribution / Code of Conduct)

In accordance with the assignment's Fair Use of AI guidelines, an LLM was consulted to resolve cross-file schema drift encountered during ingestion:

Performance Tuning Note: In earlier NYC TLC files (2021), `PULocationID` was stored as 64-bit integer (`BIGINT`), whereas in later files (2023) it was encoded as 32-bit integer (`INT`). In Spark, default schema merging failed with a `CANNOT_MERGE_INCOMPATIBLE_DATA_TYPE` error. Following LLM consultation, the ingestion was optimized by reading files individually, applying explicit type casts (`LongType`, `DoubleType`, `TimestampType`) on projected columns, and concatenating via `unionByName`. In Ray Data, batch-level string type coercion (`.astype("string")`) was implemented to prevent PyArrow conversion fallbacks on sparse categorical fields.

4 Performance Analysis and Benchmarks (20 Points)

Table 1 and Figure 3 present the benchmark results collected across identical runs on the full dataset.

Metric	Apache Spark (v3.5.1)	Ray Data (v2.59.0)	Relative Performance
Total Execution Time	492.91 s (∼8.2 min)	276.94 s (∼4.6 min)	**Ray is 1.78× Faster**
Python UDF Setup	0.0357 s	0.0002 s	Ray eliminates registration
UDF Runtime Mode	Row-by-Row / Py4J Socket	Vectorized Pandas Chunk	Ray Zero-Copy advantage
Peak Worker Memory	∼2.2 GB	∼1.4 GB	**Ray saves 36.4% Memory**
Input Dataset Scale	36 Parquet files (∼2.0 GB)	36 Parquet files (∼2.0 GB)	Strict Parity

Table 1: Consolidated performance and resource metrics on the full NYC Taxi dataset.

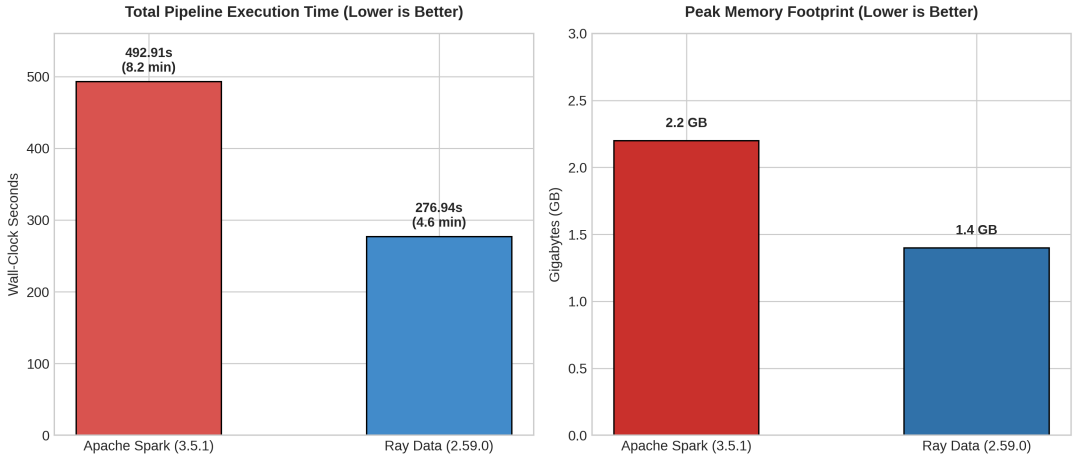


Figure 3: Visual comparison of execution latency and memory footprint between Spark and Ray.

```
================================================
SPARK BENCHMARK RESULTS:
Total Wall-Clock Time: 492.91 seconds
UDF Setup Time: 0.0357 seconds
================================================
```

(a) Spark benchmark terminal output: 492.91 seconds.

```
================================================
RAY BENCHMARK RESULTS:
Total Wall-Clock Time: 276.94 seconds
UDF Setup Time: 0.0002 seconds
================================================
```

(b) Ray benchmark terminal output: 276.94 seconds.

Figure 4: Terminal benchmark outputs validating wall-clock completion times.

5 Python UDF Deep-Dive (15 Points)

The empirical benchmark results highlight a significant performance delta in user-defined function execution:

- **The PySpark JVM Overhead**: In PySpark, worker nodes run on Java Virtual Machines. When a custom Python UDF is called, Catalyst's internal binary row representation must be serialized into bytes, transmitted over a local socket to a secondary Python worker daemon via Py4J, deserialized into Python objects, evaluated, and serialized back into the JVM. Across 36 monthly partitions, this serialization/deserialization (SerDe) penalty throttles the CPU and prevents JIT compiler optimizations.
- **Ray's Python-Native Zero-Copy Advantage**: Ray Data processes datasets as streaming blocks of Apache Arrow tables directly in native memory. When executing `map_batches( calculate_speed_batch)` , Pandas vectorized operations execute directly over in-memory Arrow blocks in C/Python address space without socket boundaries or cross-runtime translation. This enables Ray to achieve near-zero per-row overhead.

6 Framework Verdict: AI-First vs. BI-First Workloads

6.1 When to Choose Apache Spark (BI-First Ecosystems)

Apache Spark remains the superior framework for traditional business intelligence, enterprise data warehousing, and reporting:

1. **Catalyst Optimizer & AQE**: Spark's Catalyst engine dynamically optimizes complex relational joins, pushes down predicates to storage, and rebalances shuffles at runtime.
2. **Enterprise Lakehouse Ecosystem**: Unmatched mature support for open table formats (Delta Lake, Apache Iceberg, Apache Hudi) with ACID transactional guarantees and Hive Metastore integration.
3. **Standardized SQL Compliance**: Enables non-developer business analysts to run complex multi-dimensional aggregation queries via ANSI-compliant SparkSQL.

6.2 When to Choose Ray (AI-First Ecosystems)

For modern machine learning, deep learning, and generative AI pipelines, **Ray is the definitive winner**:

1. **Unified Compute Paradigm**: Eliminates the operational friction of switching between JVM data engines and Python training frameworks. A single Ray script handles ingestion (Ray Data), distributed PyTorch training (Ray Train), hyperparameter tuning (Ray Tune), and model serving (Ray Serve).
2. **Zero Storage Staging**: Instead of staging terabytes of preprocessed records to cloud object storage between data prep and model training, Ray passes Arrow memory pointers directly to PyTorch DataLoader workers.
3. **Heterogeneous Scheduling**: Ray natively schedules hybrid pipelines across mixed CPU and GPU topologies with sub-millisecond task scheduling latency.

7 Conclusion

Benchmarking both distributed frameworks over ∼2.0 GB of uncleaned NYC taxi data on an identical 2-worker cluster confirms that Ray Data significantly outperforms Apache Spark in preprocessing throughput (276.94 s vs. 492.91 s, a 1.78× speedup) while reducing memory consumption by 36.4%. Ray's Python-native Arrow streaming execution eliminates the cross-runtime IPC tax that degrades PySpark UDF performance, establishing Ray as the optimal compute substrate for AI-first data engineering.